

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Abinesh H	Reg. No.:	192424387
Section / Batch:	B.Tech AIDS / 3 rd Year	Date:	29/08/2026

Code of Conduct

I Abinesh H (192424387) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Abinesh H

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	4	4	3	3	18	92
2	4	3	4	3	4	18	
3	4	4	4	3	4	19	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: 29/08/2026	Date: _____	Date: _____

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

Introduction

An e-commerce search system can use **N-gram language models** to predict the next word entered by a customer. Bigram and trigram models capture word relationships, while backoff and deleted interpolation help handle unseen word combinations. Entropy can be used to measure the uncertainty of the predictions.

The given corpus is:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

1. Maximum Likelihood Estimate of Bigram Probability

The MLE formula for a bigram is:

$$P(\text{learning} \mid \text{machine}) = \frac{C(m, \text{achinelearning})}{C(\text{machine})}$$

Given:

- $C(m, \text{achinelearning}) = 3$
- $C(\text{machine}) = 3$

Therefore,

$$P(\text{learning} \mid \text{machine}) = \frac{3}{3} = 1$$

Interpretation

The probability is **1 or 100%**. This means that every occurrence of **machine** in the training corpus is followed by **learning**.

Therefore, when a customer types:

"machine"

the system strongly predicts:

learning

This improves autocomplete accuracy for frequently observed product-related search phrases.

2. Backoff for the Unseen Sequence

The customer enters:

"machine learning transforms"

The trigram:

$(m, a, \text{chinelearningtransforms})$

does not occur in the corpus.

The system therefore backs off to the bigram:

$$P(\text{transforms} \mid \text{learning})$$

But "**learning transforms**" is also not present.

Finally, it checks the unigram probability:

$$P(\text{transforms})$$

The word **transforms** does not occur in the corpus.

Therefore:

$$P(\text{transforms} \mid \text{learning}) = 0$$

under the given **unsmoothed** corpus.

Why Backoff is Useful

Backoff is useful because real users frequently enter search queries that were not present in the training data.

Instead of immediately failing at the trigram level, the system attempts to use lower-order information.

In a larger real-world system with a sufficiently large vocabulary, the unigram level can provide a non-zero fallback probability. Smoothing can further ensure that completely unseen words do not receive zero probability.

3. Deleted Interpolation Probability

For:

machine learning improves

the required probabilities are:

Trigram

$$P(i, mproves \mid machinelearning) = \frac{1}{3} \approx 0.33$$

Bigram

$$P(improves \mid learning) = \frac{1}{3} \approx 0.33$$

Unigram

There are 12 total words in the corpus, and **improves** occurs once.

$$P(improves) = \frac{1}{12} \approx 0.0833$$

Given:

- $\lambda_1 = 0.5$
- $\lambda_2 = 0.3$
- $\lambda_3 = 0.2$

The interpolation formula is:

$$\begin{aligned}P &= 0.5P_{tri} + 0.3P_{bi} + 0.2P_{uni} \\P &= (0.5)(0.33) + (0.3)(0.33) + (0.2)(0.0833) \\P &\approx 0.165 + 0.099 + 0.0167 \\ \boxed{P \approx 0.2807}\end{aligned}$$

Interpretation

Deleted interpolation combines information from three levels. Even if the trigram is rare or unavailable, the bigram and unigram models can contribute to the final probability. This makes the prediction system more robust when training data is sparse.

4. Entropy of the Prediction Distribution

Given:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Entropy is:

$$H = - \sum p(x) \log_2 p(x)$$

Therefore:

$$\begin{aligned}H &= -[3(0.33 \log_2(0.33))] \\ \boxed{H \approx 1.58 \text{ bits}}\end{aligned}$$

The maximum entropy for three equally probable outcomes is:

$$\log_2(3) \approx 1.585$$

Thus, the prediction distribution has **very high uncertainty** because all three words are almost equally likely.

Interpretation

The search system has no strong preference among **improves, enables, and drives**. More training information or additional context would be required to make the prediction more confident.

5. Python Implementation

```
import nltk
import numpy as np
import math
from nltk.util import ngrams
from collections import Counter

corpus = """machine learning improves business,
machine learning enables automation,
machine learning drives innovation"""

# Tokenization
words = nltk.word_tokenize(corpus.lower())

# N-grams
uni = Counter(words)
bi = Counter(ngrams(words, 2))
tri = Counter(ngrams(words, 3))
print("UNIGRAMS:", uni)
print("\nBIGRAMS:", bi)
print("\nTRIGRAMS:", tri)

# 1. MLE Bigram
p_bigram = bi[("machine", "learning")] / uni["machine"]
print("\nP(learning | machine) =", p_bigram)

# 2. Backoff
word = "transforms"
p_tri = tri[("machine", "learning", word)]
p_bi = bi[("learning", word)]
p_uni = uni[word]
print("\nBACKOFF:")
print("Trigram count =", p_tri)
print("Bigram count =", p_bi)
print("Unigram count =", p_uni)
print("Backoff probability =", 0)

# 3. Deleted Interpolation
p_tri = 1 / 3
p_bi = 1 / 3
p_uni = 1 / len(words)
```

```

final_p = 0.5*p_tri + 0.3*p_bi + 0.2*p_uni
print("\nINTERPOLATION:")
print("Trigram =", p_tri)
print("Bigram =", p_bi)
print("Unigram =", p_uni)
print("Final probability =", final_p)
# 4. Entropy
probs = np.array([0.33, 0.33, 0.33])
entropy = -sum(p * math.log2(p) for p in probs)
print("\nEntropy =", entropy, "bits")
# Final prediction
print("\nTop prediction after 'learning':")
print("improves, enables, drives")

```

Conclusion

The N-gram model successfully predicts words based on previously observed sequences. The MLE probability of **learning after machine is 1**, showing a strong relationship in the corpus. Backoff provides a fallback mechanism for unseen sequences, while interpolation combines different levels of information. The entropy of approximately **1.58 bits** shows that the three possible predictions are almost equally uncertain.

Output:

```

UNIGRAMS: Counter({'machine': 3, 'learning': 3, ',': 2, 'improves': 1, 'business': 1, 'enables': 1, 'automation': 1,
'drives': 1, 'innovation': 1})
BIGRAMS: Counter({'('machine', 'learning)': 3, ('(', 'machine)': 2, ('learning', 'improves)': 1, ('improves',
'business)': 1, ('business', ','): 1, ('learning', 'enables)': 1, ('enables', 'automation)': 1, ('automation', ','): 1, ('learning',
'drives)': 1, ('drives', 'innovation)': 1})
TRIGRAMS: Counter({'(', '(', 'machine', 'learning)': 2, ('machine', 'learning', 'improves)': 1, ('learning', 'improves',
'business)': 1, ('improves', 'business', ','): 1, ('business', '(', 'machine)': 1, ('machine', 'learning', 'enables)': 1,
('learning', 'enables', 'automation)': 1, ('enables', 'automation', ','): 1, ('automation', '(', 'machine)': 1, ('machine',
'learning', 'drives)': 1, ('learning', 'drives', 'innovation)': 1})

```

```

P(learning | machine) = 1.0

BACKOFF:
Trigram count = 0
Bigram count = 0
Unigram count = 0
Backoff probability = 0

INTERPOLATION:
Trigram = 0.3333333333333333
Bigram = 0.3333333333333333
Unigram = 0.07142857142857142
Final probability = 0.28095238095238095

Entropy = 1.5834674497121084 bits

Top prediction after 'learning':
improves, enables, drives
[nltk_data] Unzipping tokenizers/punkt_tab.zip.

```

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

Introduction

A healthcare chatbot must understand the grammatical role of words before identifying the user's intention. POS tagging helps distinguish words with multiple meanings. The word **book** can function as either a verb or a noun depending on its context.

The two sentences are:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

1. Penn Treebank POS Tagging

Sentence 1

Book an appointment with the doctor.

Word	POS Tag	Meaning
Book	VB	Verb
an	DT	Determiner
appointment	NN	Noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Noun

Here, **book** is a **verb (VB)** because it expresses an action or request.

The sentence means that the user wants to **schedule an appointment**.

Sentence 2

The book contains medical information.

Word	POS Tag	Meaning
The	DT	Determiner
book	NN	Noun
contains	VBZ	Verb
medical	JJ	Adjective
information	NN	Noun

Here, **book** is a **noun (NN)** because it is an object referred to by the determiner **the**.

2. HMM Probability Calculation

Given:

$$P(\textit{book} \mid \textit{VB}) = 0.7$$

$$P(\textit{book} \mid \textit{NN}) = 0.3$$

and:

$$P(\textit{Start} \rightarrow \textit{VB}) = 0.6$$

$$P(\textit{Start} \rightarrow \textit{NN}) = 0.4$$

The probability of each tag sequence is calculated as:

Verb

$$P(V, B\textit{book}) = P(\textit{VB} \mid \textit{Start})P(\textit{book} \mid \textit{VB})$$

$$= 0.6 \times 0.7$$

$$\boxed{0.42}$$

Noun

$$P(N, N\textit{book}) = P(\textit{NN} \mid \textit{Start})P(\textit{book} \mid \textit{NN})$$

$$= 0.4 \times 0.3$$

$$\boxed{0.12}$$

Since:

$$0.42 > 0.12$$

the HMM chooses:

$$\boxed{\textit{VB}}$$

Interpretation

At the beginning of a sentence, the given probabilities favor **VB** because both the emission probability and the start-transition probability are higher for VB.

3. Rule-Based vs Stochastic HMM Tagging

Feature	Rule-Based	Stochastic HMM
Method	Hand-written rules	Probability-based
Training data	Not always required	Required
Context	Uses grammatical rules	Uses probabilities
Interpretability	High	Moderate
Flexibility	Lower	Higher
Ambiguity	Depends on rules	Can calculate most likely tag

Rule-Based Approach

A rule-based system could use:

If **book** occurs at the beginning as an instruction, tag it VB.

It is simple and easy to understand but may fail with unusual sentence structures.

HMM Approach

The HMM calculates probabilities from training data and selects the most probable tag. It can handle ambiguity statistically, but its performance depends on the quality and size of the training corpus.

Healthcare Application

For a healthcare chatbot, accurate tagging is important because:

"Book an appointment"

indicates an **appointment-booking intent**, while:

"The book contains medical information"

does not represent an appointment request.

4. Importance of Standardized POS Tagsets

The Penn Treebank tagset provides standardized grammatical labels such as:

- NN – noun
- VB – verb
- VBZ – third-person singular verb
- JJ – adjective
- DT – determiner
- IN – preposition

These standardized tags support:

Intent Detection

The verb **book** helps identify an appointment-booking action.

Entity Identification

Nouns such as **doctor**, **appointment**, and medical terms can assist in identifying important entities.

Response Generation

Understanding grammatical structure helps the chatbot generate an appropriate response.

Thus, POS tagging acts as an important intermediate step between raw text and higher-level NLP tasks.

5. Python Implementation

```
import nltk

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

for sentence in sentences:
    tokens = nltk.word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)
    print("\nSentence:", sentence)
    print("Tokens:", tokens)
    print("POS Tags:", tags)

# HMM probabilities
emission = {"VB": 0.7, "NN": 0.3}
transition = {"VB": 0.6, "NN": 0.4}
vb = emission["VB"] * transition["VB"]
nn = emission["NN"] * transition["NN"]
print("\nHMM CALCULATION")
print("P(book, VB) =", vb)
print("P(book, NN) =", nn)
if vb > nn:
    print("Most probable tag: VB")
else:
    print("Most probable tag: NN")
```

Conclusion

POS tagging allows the chatbot to distinguish the two meanings of **book**. The HMM calculation gives a probability of **0.42 for VB** and **0.12 for NN**, so VB is preferred at the beginning of a sentence using the supplied probabilities. Standardized POS tags can then support intent detection, entity recognition, and response generation.

Output:

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.

Sentence: Book an appointment with the doctor.
Tokens: ['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']
POS Tags: [('Book', 'NNP'), ('an', 'DT'), ('appointment', 'NN'), ('with', 'IN'), ('the', 'DT'), ('doctor', 'NN'), ('.', '.')]

Sentence: The book contains medical information.
Tokens: ['The', 'book', 'contains', 'medical', 'information', '.']
POS Tags: [('The', 'DT'), ('book', 'NN'), ('contains', 'VBZ'), ('medical', 'JJ'), ('information', 'NN'), ('.', '.')]

HMM CALCULATION
P(book, VB) = 0.42
P(book, NN) = 0.12
Most probable tag: VB
```

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

Introduction

Financial news systems process large volumes of text and require accurate POS tagging. Statistical taggers can make errors, so a **Transformation-Based Tagger** can apply learned rules to correct them.

The sentence is:

"Market growth drives investment."

Initial tags:

market/NN

growth/NN

drives/NNS

investment/NN

Transformation rule:

Change NNS to VBZ if the preceding word is tagged as NN.

1. Applying the Transformation Rule

Initial sequence:

market/NN growth/NN drives/NNS investment/NN

The word **drives** is tagged NNS.

Its preceding word is:

growth/NN

Since the preceding word has tag NN, the transformation rule applies:

NNS → VBZ

Corrected Sequence

market/NN

growth/NN

drives/VBZ

investment/NN

Why is this appropriate?

The sentence structure is:

Market growth drives investment.

Here, **market growth** is the subject and **drives** is the main verb. Since the subject is singular, the verb takes the third-person singular form **drives**, which corresponds to VBZ.

2. Analysis of the Tagging Error

The initial tagger classified:

drives → NNS

NNS represents a plural noun.

However, in:

Market growth drives investment.

the word **drives** describes an action performed by **market growth**.

Therefore:

drives → VBZ

The surrounding context is important:

Market/NN growth/NN drives/?

The two nouns form the subject, followed by the action performed by that subject. This makes VBZ more appropriate than NNS.

3. Relative Word Frequencies

Given:

Word	Frequency
market	500
growth	350
drives	180
investment	420

Total frequency:

$$500 + 350 + 180 + 420 = 1450$$

Relative frequency:

$$P(word) = \frac{Frequency(word)}{1450}$$

Calculations

Market

$$P(\text{market}) = \frac{500}{1450} = 0.3448$$

Growth

$$P(\text{growth}) = \frac{350}{1450} = 0.2414$$

Drives

$$P(\text{drives}) = \frac{180}{1450} = 0.1241$$

Investment

$$P(\text{investment}) = \frac{420}{1450} = 0.2897$$

Frequency Distribution

Word	Probability
------	-------------

market	0.3448
--------	--------

growth	0.2414
--------	--------

drives	0.1241
--------	--------

investment	0.2897
------------	--------

These probabilities can help a statistical POS tagger estimate how frequently words occur and combine lexical information with contextual tag probabilities.

4. Entropy Before and After Transformation

Entropy is:

$$H = - \sum P(w) \log_2 P(w)$$

Using the four probabilities:

$$H = -[0.3448 \log_2(0.3448) + 0.2414 \log_2(0.2414) + 0.1241 \log_2(0.1241) + 0.2897 \log_2(0.2897)]$$

Therefore:

$$H \approx 1.916 \text{ bits}$$

Before Transformation

$$H_{before} \approx 1.916 \text{ bits}$$

After Transformation

The transformation changes only the POS tag of **drives**. It does **not change the word frequencies**.

Therefore:

$$H_{after} \approx 1.916 \text{ bits}$$

Interpretation

The word-frequency distribution remains unchanged, so its entropy also remains unchanged.

This is important: **POS correction improves tagging accuracy but does not change lexical word-frequency uncertainty**. If we wanted to measure uncertainty specifically in POS tags, we would need a distribution of POS-tag frequencies before and after correction.

5. Python Implementation

```
import nltk
import math
from collections import Counter
sentence = "Market growth drives investment."
# Tokenization
tokens = nltk.word_tokenize(sentence)
# Given initial tags
initial = [
    ("market", "NN"),
    ("growth", "NN"),
    ("drives", "NNS"),
    ("investment", "NN")
]
print("Initial POS Tags:")
print(initial)
# Transformation-Based correction
corrected = initial.copy()
```

```

for i in range(1, len(corrected)):
    word, tag = corrected[i]
    previous_tag = corrected[i-1][1]
    if tag == "NNS" and previous_tag == "NN":
        corrected[i] = (word, "VBZ")
print("\nCorrected POS Tags:")
print(corrected)
# Word frequencies
freq = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}
total = sum(freq.values())
print("\nWORD FREQUENCIES")
probabilities = {}
for word, count in freq.items():
    p = count / total
    probabilities[word] = p
    print(word, ":", count, "Probability =", round(p, 4))
# Entropy
entropy = -sum(
    p * math.log2(p)
    for p in probabilities.values()
)
print("\nEntropy before transformation =", round(entropy, 4))
print("Entropy after transformation =", round(entropy, 4))
print("\nFinal POS Tagging Result:")
print(corrected)

```

Conclusion

The transformation-based tagger successfully changes **drives/NNS** to **drives/VBZ** because it follows two preceding contextual conditions: the original tag is NNS and the preceding word is tagged NN. The corpus contains a total of **1450 word occurrences**, and the calculated word-frequency entropy is approximately **1.916 bits**. Since the transformation changes only POS tags and not word frequencies, the word-frequency entropy remains unchanged.

Output:

```
Initial POS Tags:
[('market', 'NN'), ('growth', 'NN'), ('drives', 'NNS'), ('investment', 'NN')]

Corrected POS Tags:
[('market', 'NN'), ('growth', 'NN'), ('drives', 'VBZ'), ('investment', 'NN')]

WORD FREQUENCIES
market : 500 Probability = 0.3448
growth : 350 Probability = 0.2414
drives : 180 Probability = 0.1241
investment : 420 Probability = 0.2897

Entropy before transformation = 1.9161
Entropy after transformation  = 1.9161

Final POS Tagging Result:
[('market', 'NN'), ('growth', 'NN'), ('drives', 'VBZ'), ('investment', 'NN')]
```